Module 4: Lesson 2

RNN implementation

Outline

- Implementing an RNN
- Predicting stock prices with RNNs

Implementing a RNN in practice

In lesson 2, we will implement an RNN in practice for the first time. As you will see in the accompanying notebook, most of the process (at least from a coding perspective) is identical to what we have done before for other neural network architectures:

1. Gather data
2. Define samples: validation, train, and test
   → Note here that we will also need to define the 'window' for the sequence of inputs we want to consider!
3. Defining the model architecture

```
model = tf.keras.models.Sequential([
  tf.keras.layers.SimpleRNN(50, return_sequences=True, input_shape=(X_train.shape[1], n_features)),
  tf.keras.layers.Dropout(0.2),
  tf.keras.layers.SimpleRNN(50, return_sequences=True),
  tf.keras.layers.Dropout(0.2),
  tf.keras.layers.SimpleRNN(50, return_sequences=True),
  tf.keras.layers.Dropout(0.2),
  tf.keras.layers.SimpleRNN(50,return_sequences=False),
  tf.keras.layers.Dropout(0.2),
  tf.keras.layers.Dense(10),
  tf.keras.layers.Dense(1)
])
```

4. Training the model!

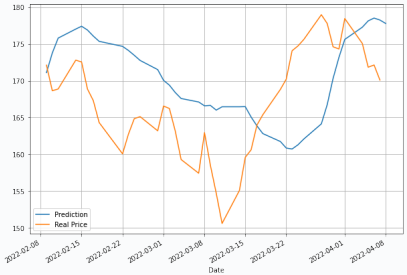
Predicting stock prices with RNNs

We will practice our implementation of RNNs with a prediction problem already familiar to us: **future evolution of a stock!**

- In this case, we will predict **prices** (rather than returns). Indeed, it makes sense that prices follow a sort of 'sequence' so that we can predict the next day's price by learning the sequence with RNNs.
- We build an RNN model to predict $d + 1$ price given a sequence of up to 20 past obs. on input prices.

Layer (type)	Output Shape	Param #
simple_rnn (SimpleRNN)	(None, 20, 50)	2600
dropout (Dropout)	(None, 20, 50)	0
simple_rnn_1 (SimpleRNN)	(None, 20, 50)	5050
dropout_1 (Dropout)	(None, 20, 50)	0
simple_rnn_2 (SimpleRNN)	(None, 20, 50)	5050
dropout_2 (Dropout)	(None, 20, 50)	0
simple_rnn_3 (SimpleRNN)	(None, 50)	5050
dropout_3 (Dropout)	(None, 50)	0
dense (Dense)	(None, 10)	510
dense_1 (Dense)	(None, 1)	11

Total params: 18,271
Trainable params: 18,271
Non-trainable params: 0



Summary of Lesson 2

In Lesson 2, we:

- Implemented our first RNN
- Tested the performance of RNNs in predicting stock prices

⇒ **References for this lesson:**

Chapter 9, Sections 9.5 and 9.6 of Zhang, Ashton, et al. *Dive into Deep Learning*.

TO DO NEXT: Go to the accompanying Python Notebook to implement your first RNN!

In the next lesson, we will improve different aspects of the network and also explore in depth the details of forward and backpropagation in RNNs.